



React Router II





**Activen las cámaras los que puedan y
pasemos asistencia**

***Crear una SPA que utilice
parámetros de la URL para la
renderización de una vista
detalle accedida por
redireccionamiento
programático.***

{desafío}
latam_

- Unidad 1: Introducción a React.
- Unidad 2: Estados de los componentes y eventos.
- Unidad 3: Renderización dinámica de componentes.
- Unidad 4: Consumo de APIs con React.
- Unidad 5: React Router I
- Unidad 6: Context
- Unidad 7: React Router II
- Unidad 8: JWT



Te encuentras aquí





Inicio

{desafío}
latam_



/ Crear un menú de navegación usando el componente NavLink para detectar y estilizar la ruta activa*/*

/ Redireccionar programáticamente a un usuario usando el hook useNavigate */*

/ Obtener los parámetros de una ruta en un componente usando el hook useParams()*/*

/ Proteger rutas en base a una sesión de usuario activa */*

Objetivos

Activación de conceptos

Contesta la pregunta correctamente y gana un punto

Instrucciones:

- Se realizará una pregunta, el primero en escribir “YO” por el chat, dará su respuesta al resto de la clase.
- El docente validará la respuesta.
- En caso de que no sea correcta, dará la oportunidad a la segunda persona que dijo “Yo”.
- Cada estudiante podrá participar un máximo de 2 veces.
- Al final, el/la docente indicará el 1º, 2º y 3º lugar.
- Esta actividad no es calificada, es solo una dinámica para recordar los conceptos clave para abordar esta sesión.





Activación de conceptos



¿Cómo podemos pasar un estado a un
componente hijo ?



Activación de conceptos



¿En qué consiste el problema de prop drilling?



Activación de conceptos



¿Para qué sirve Context?



Activación de conceptos



¿Cuál es el rol del componente Provider?



Activación de conceptos



¿Para qué sirve useContext?



Activación de conceptos



Primer lugar:



Segundo lugar:

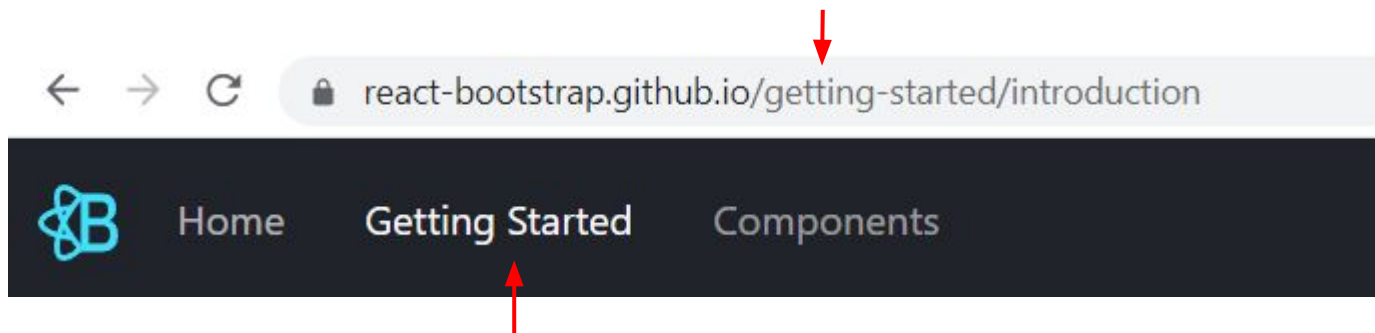


Tercer lugar:

React Router

La clase active

Existen consideraciones visuales que aportan a una mejor experiencia de usuario, entre ellas, personalizar el estilo de la opción que representa la ruta actual.



Este pequeño, pero importante cambio visual, ayudará a nuestros usuarios a estar mejor ubicados mientras navegan en nuestra aplicación.

React Router

El componente NavLink

React Router nos provee un componente que nos ayudará a saber cuál de las opciones de navegación corresponde a la ruta consultada.

```
<NavLink
  className={ ({ isActive }) => (isActive ? "nombre-de-la-clase" : undefined) }
  to="/" >
  Home
</NavLink>
```

A diferencia del ya conocido componente Link, este componente tiene integrado un atributo **isActive** que nos devolverá **true** o **false** si la ruta está siendo consultada.

React Router

El componente NavLink

Para acceder a este atributo, necesitamos definir una función como valor del atributo `className`, la cual tendrá como parámetro un objeto con la propiedad **`isActive`** y deberá retornar el nombre de la clase que será asignada a este elemento.

Para ver en la práctica el uso de este componente, crea una nueva aplicación de React e instala React Router y realicemos los siguientes pasos:

1) Agrega una clase `.active` en **`index.css`**

```
.App {  
  font-family: sans-serif;  
  text-align: center;  
}  
  
.active {  
  color: darkgreen;  
}
```


React Router

El componente NavLink

2) Crea un componente **Navbar** con 2 opciones: Home y Personajes.

```
import { NavLink } from "react-router-dom";

export default function Navbar() {
  const setActiveClass = ({ isActive }) => (isActive ? "active" : undefined);
  return (
    <div>
      <NavLink className={ setActiveClass } to="/">
        Home
      </NavLink>
      {" - "}
      <NavLink className={ setActiveClass } to="/personajes">
        Personajes
      </NavLink>
    </div>
  );
}
```

React Router

El componente NavLink

3) Modifica el código de **App.jsx** para agregar el enrutador e incluir el Navbar creado.

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Navbar from "../components/Navbar";

export default function App() {
  return (
    <div className="App">
      <BrowserRouter>
        <Navbar />
      </BrowserRouter>
    </div>
  );
}
```

React Router

El componente NavLink

El resultado de lo anterior lo podremos apreciar levantando la aplicación y visitando las rutas.

← → ↻ dljebf.csb.app

Home - Personajes

← → ↻ dljebf.csb.app/personajes

Home - Personajes

Ejercicio

Continuando con nuestra aplicación:

1. Agrega una nueva opción en el Navbar.
2. Crea una nueva clase para definir los estilos de la ruta activa.
3. Crea una nueva clase para definir los estilos de las rutas que no estén activas.
4. Comprueba que las nuevas clases son asignadas correctamente al consultar las rutas.

Ejercicio

¡Manos al teclado!



*/** Crear un menú de navegación usando el componente NavLink para detectar y estilizar la ruta activa**/* ✓

*/** Redireccionar programáticamente a un usuario usando el hook useNavigate **/*

*/** Obtener los parámetros de una ruta en un componente usando el hook useParams()**/*

*/** Proteger rutas en base a una sesión de usuario activa **/*

Objetivos

React Router

El hook `useNavigate()`

En algunos casos podríamos necesitar redireccionar a nuestros usuarios sometiendo esta acción a una condición o simplemente agregar en la URL de destino alguna variable.

A esto se le conoce como un redireccionamiento programático y en React Router podemos realizarlo utilizando el hook **`useNavigate()`**



React Router

El hook `useNavigate()`

Veamos un ejemplo del redireccionamiento programático, creando una vista que reciba del usuario un id numérico y redireccione a una ruta `/personajes/:id`

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";

export default function Home() {
  const [id, setId] = useState("");
  const navigate = useNavigate();
  const irAPersonajes = () => {
    navigate(`/personajes/${id}`);
  };
  return (
    <div className="mt-5">
      <h1>React Router II</h1>
      <input
        type="number"
        value={id}
        onChange={({ target }) => setId(target.value)}
      />
      <button onClick={ irAPersonajes }>buscar</button>
    </div>
  );
}
```

React Router

El hook useNavigate()

Ahora crearemos una vista Personajes que simplemente retorne un `OK`

```
export default function Personajes() {  
  return (  
    <div className="mt-5">  
      <h1>OK</h1>  
    </div>  
  );  
}
```


React Router

El hook useNavigate()

Finalmente, hay que agregar en el componente App las rutas relacionadas a las vistas creadas.

```
<BrowserRouter>
  <Navbar />

  <Routes>

    <Route path="/" element={<Home />} />

    <Route path="/personajes/:id" element={<Personajes />} />

  </Routes>

</BrowserRouter>
```

Recuerda importar las vistas al inicio del archivo.

React Router

El hook `useNavigate()`

Probemos nuestra aplicación escribiendo un id en el input y presionando el botón.

← → ↻ dljebf.csb.app

[Home](#) - [Personajes](#)

React Router II

La URL de destino incluye el id escrito en la vista Home

← → ↻

dljebf.csb.app/personajes/5

[Home](#) - [Personajes](#)



Ejercicio

Continuando con nuestra aplicación:

1. Cambia el input por un selector del 1 al 10.
2. Condiciona el redireccionamiento en función de que la variable **id** contenga algún valor.
3. Comprueba los cambios en el navegador.

Ejercicio

¡Manos al teclado!



/* Crear un menú de navegación usando el componente NavLink para detectar y estilizar la ruta activa */ ✓

/* Redireccionar programáticamente a un usuario usando el hook useNavigate */ ✓

/* Obtener los parámetros de una ruta en un componente usando el hook useParams() */

/* Proteger rutas en base a una sesión de usuario activa */

Objetivos

React Router

El hook `useParams()`

Con React Router podemos definir en el ***path*** de una ruta el reconocimiento de parámetros.

```
<Route path="/categoria/:nombre" element={<Componente />} />
```

Por ejemplo, si se consulta: `/categoria/pib`

El parámetro **nombre** tomará como valor: `"pib"`

React Router

El hook `useParams()`

- Para que un componente acceda a los parámetros de la URL, se necesita ocupar el hook ***`useParams()`***
- Al ser ejecutado, este hook nos devuelve un objeto con todos los parámetros.
- Veamos un ejemplo capturando el **`id`** del personaje que logramos incluir en la URL a través de un redireccionamiento programático.

React Router

El hook `useParams()`

```
import { useParams } from "react-router-dom";

export default function Personajes() {

  const { id } = useParams();

  return (
    <div className="mt-5">
      <h1>{ id }</h1>
    </div>
  );
}
```

Modifica el código de la vista *Personajes* para que reciba y muestre el parámetro en lugar del 

React Router

El hook `useParams()`

Si probamos nuevamente el redireccionamiento programático, ahora veremos que se visualiza el valor del id en el componente.



dljebf.csb.app/personajes/222

Home - Personajes

222

Ejercicio

En la vista Personajes:

1. Utiliza el **id** recibido en parámetros para consultar la [API de Rick And Morty](#) y obtener la información del personaje que coincida con el id.
2. Agrega React Bootstrap al proyecto para ocupar el componente Card y mostrar la información del personaje.
3. Condiciona el componente Card a solo mostrarse en caso que haya obtenido un personaje con el id desde la vista Home, de lo contrario, muestra en el componente un texto indicando que no se encontró ningún personaje con el id ingresado.

Ejercicio ¡Manos al teclado!



/* Crear un menú de navegación usando el componente NavLink para detectar y estilizar la ruta activa */ ✓

/* Redireccionar programáticamente a un usuario usando el hook useNavigate */ ✓

/* Obtener los parámetros de una ruta en un componente usando el hook useParams() */ ✓

/* Proteger rutas en base a una sesión de usuario activa */

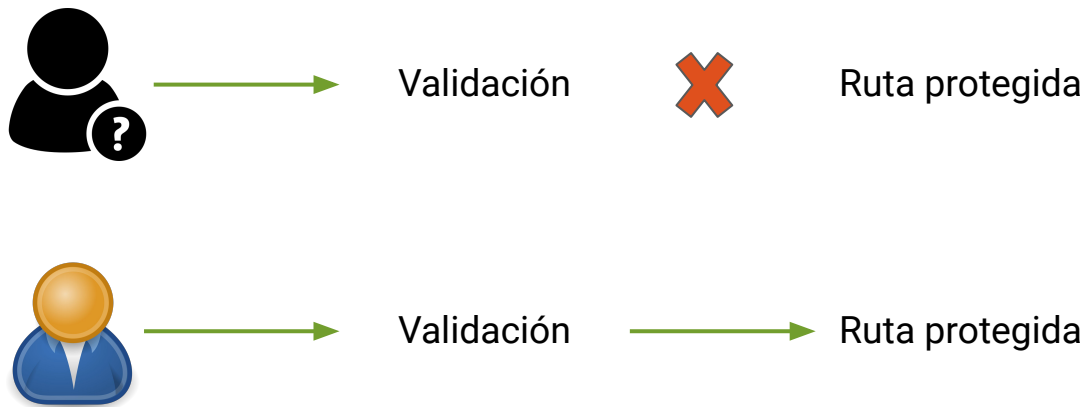
Objetivos

React Router

Rutas protegidas

Cuando desarrollamos sistemas que incluyan cuentas de usuario, normalmente tendremos secciones restringidas a solo aquellas que tengan una sesión de usuario activa.

Para lograr esto podemos condicionar la ruta protegida para que valide la existencia previa de una sesión antes de renderizar su componente en cuestión.



React Router

Rutas protegidas

Llevemos este caso a la práctica con una pequeña aplicación que contenga 3 rutas:

- Home
- Administración
- Login

Siendo la **Administración** la ruta protegida o privada, mientras que **Home** y **Login** serán rutas que denominaremos como públicas.

Crea un nuevo proyecto de React e instala React Router DOM para realizar el siguiente ejercicio.

React Router

Rutas protegidas

Iniciemos creando nuestro Navbar. Para eso utiliza lo siguiente.

```
nav {  
  display: flex;  
  justify-content: center;  
  gap: 10px;  
  padding: 30px;  
}  
  
.active {  
  color: darkgreen;  
  font-weight: bold;  
}
```

```
const Navbar = () => {  
  
  const setActiveClass = ({ isActive }) => (isActive ? "active" : undefined);  
  
  return (  
    <div>  
      <nav>  
  
        <NavLink className={setActiveClass} to="/"> Home </NavLink>  
  
        <NavLink className={setActiveClass} to="/admin"> Administración </NavLink>  
  
        <NavLink className={setActiveClass} to="/login"> Login </NavLink>  
  
      </nav>  
    </div>  
  );  
};
```

React Router

Rutas protegidas

Ahora creemos nuestros 3 componentes relacionados a las 3 rutas de nuestro navbar.

```
const Administracion = () => {  
  return <div>Administración</div>;  
};  
export default Administracion;
```

```
const Home = () => {  
  return <div>Home</div>;  
};  
export default Home;
```

```
const Login = () => {  
  return <div>Login</div>;  
};  
export default Login;
```

React Router

Rutas protegidas

Creamos el **userContext** con un usuario que por defecto estará en null y utilizaremos su provider en main.jsx

```
import { createContext, useState } from "react";

export const UserContext = createContext();

const UserProvider = ({ children }) => {
  const [user, setUser] = useState(null);

  return (
    <UserContext.Provider value={{ user, setUser }}>
      {children}
    </UserContext.Provider>
  );
};

export default UserProvider;
```

{desafío}
latam_

```
import UserProvider from "../context/userContext.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <UserProvider>
        <App />
      </UserProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

React Router

Rutas protegidas

```
const App = () => {  
  const { user } = useContext(UserContext);  
  
  return (  
    <div>  
      <Navbar />  
      <Routes>  
        <Route  
          path="/" />  
          element={<Home />}  
        />  
        <Route  
          path="/admin"  
          element={user ? <Administracion /> : <Navigate to="/login" />}  
        />  
        <Route  
          path="/login"  
          element={<Login />}  
        />  
      </Routes>  
    </div>  
  );  
};  
export default App;
```

Aquí es donde
ocurre la magia



Finalmente, solo nos falta el código del componente App y un pequeño contexto que simula la aplicación de un sistema de usuarios en nuestra aplicación.

React Router

Rutas protegidas

La protección de la ruta **/admin** consiste en validar el valor de la constante **user** para decidir renderizar el componente **Administración** o de lo contrario, usar el componente **Navigate** de React Router DOM para redireccionar el usuario a **/login**

```
<Route path="/admin" element={user ? <Administracion /> : <Navigate to="/login" />} />
```



user puede ser finalmente cualquier variable o expresión que quisiéramos usar para condicionar el acceso de esta ruta

React Router

Rutas protegidas

Ahora probemos nuestra aplicación en el navegador y veremos que al intentar acceder a la ruta del componente **Administración**, somos redireccionados inmediatamente a la ruta del componente **Login**



[Home](#) [Administracion](#) [Login](#)

Ingresar a tu cuenta....

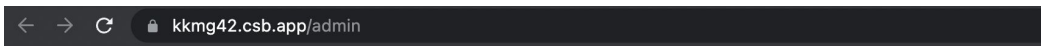
Este redireccionamiento es efectivo incluso ingresando desde la URL a la ruta protegida.

React Router

Rutas protegidas

En este ejemplo, para poder acceder a la Administración, entonces basta con asignarle un valor a la propiedad **user** del contexto.

```
const [user, setUser] = useState({  
  email: "test@desafiolatam.com",  
  displayName: "Desafío Latam",  
});
```



[Home](#) [Administracion](#) [Login](#)

Bienvenido a la administración

Ejercicio

Continúa el ejercicio anterior sobre rutas protegidas, agregando la posibilidad de registrar usuarios y poder iniciar sesión.

Usa tu creatividad para agregar las funcionalidades que creas correspondientes.

Ejercicio

¡Manos al teclado!



/* Crear un menú de navegación usando el componente NavLink para detectar y estilizar la ruta activa */ ✓

/* Redireccionar programáticamente a un usuario usando el hook useNavigate */ ✓

/* Obtener los parámetros de una ruta en un componente usando el hook useParams() */ ✓

/* Proteger rutas en base a una sesión de usuario activa */ ✓

Objetivos



Cierre

{desafío}
latam_



¿Existe algún concepto que no
hayas comprendido?

Reflexionemos

- Revisemos el material de estudio asincrónico en donde repasamos los conceptos de esta clase.
- Revisemos el desafío de la unidad leyendo la descripción y requerimientos.

A vertical line separates the white left side from the blue right side. It is decorated with a series of white and grey symbols: a closing curly brace '}', an '@' symbol, an opening curly brace '{', a smiley face ':)', and a large 'Z' or '7' shape.

¿Qué sigue?

¿Tienen alguna duda respecto al Desafío?



*Academia de
talentos digitales*

www.desafiolatam.com



/DesafioLatam



/DesafioLatam



/DesafioLatam



/DesafioLatam